Practical 6: Arrays, Strings, Linked Lists, Stacks and Queues

Aim

To study and implement basic operations on arrays, strings, linked lists, stacks, and queues in Python.

Theory

Arrays (implemented as lists in Python) store elements in contiguous, indexable memory locations, giving O(1) random access by index; appending to the end is amortized O(1), while inserting/deleting at arbitrary positions is O(n) due to element shifting. Strings in Python are immutable sequences of characters, meaning any operation that appears to modify a string (such as reversal) actually creates and returns a new string object; slicing with a step of -1 (s[::-1]) is a concise idiom for reversal because it walks the sequence from the last index to the first. A Stack enforces LIFO (Last In First Out) order, supporting push (insert at top) and pop (remove from top) in O(1) time, useful for reversing sequences, expression parsing, and recursion simulation. A Queue enforces FIFO (First In First Out) order, supporting enqueue (insert at rear) and dequeue (remove from front); collections.deque is preferred over a plain list for queues because it is backed by a doubly linked structure, giving O(1) complexity for operations at both ends versus O(n) for list.pop(0).

Algorithm

1. Create a list (array) and append a new element to it.
2. Reverse a given string using slicing (s[::-1]).
3. Create a stack using a list, extend it with elements, and pop the last element.
4. Create a queue using deque, initialize it with elements, and dequeue from the front.
5. Print the results after each operation.

Program

```
arr = [5, 2, 8, 1]
arr.append(9)
print("Array:", arr)

s = "Hello World"
print("Reversed String:", s[::-1])

stack = []
stack.extend([1, 2, 3])
stack.pop()
print("Stack after pop:", stack)

from collections import deque
queue = deque([1, 2, 3])
queue.popleft()
print("Queue after dequeue:", list(queue))
```

Output

```
Array: [5, 2, 8, 1, 9]
Reversed String: dlroW olleH
Stack after pop: [1, 2]
Queue after dequeue: [2, 3]
```

Result / Conclusion

Basic operations on arrays, strings, stacks, and queues were successfully implemented and verified.

Viva Questions

Q1. What is the difference between a Python list and a traditional array?

Ans: A traditional array (e.g., in C) stores elements of the same data type in contiguous memory, providing fixed size and direct memory access. Python lists are dynamic arrays that can store heterogeneous types (int, str, objects), automatically resize, and support O(1) append (amortised). Python's array module or NumPy arrays provide typed arrays closer to traditional ones. For numeric computation, NumPy arrays are more memory-efficient and faster.

Q2. Why are Python strings immutable, and what are the implications?

Ans: Strings are immutable because Python interns (caches) string objects for memory efficiency and thread safety — multiple variables can safely reference the same string object. Implications: any 'modification' (concatenation, slicing, upper()) creates a new string object. For performance when building strings in a loop, use a list and ''.join(parts) instead of repeated += which creates O(n^2) temporary objects.

Q3. What does the slicing notation s[::-1] do?

Ans: s[start:stop:step] is Python's slice syntax. s[::-1] uses start=None (end of string), stop=None (beginning of string), and step=-1 (move backwards one character at a time). This traverses the string from the last character to the first, effectively reversing it. It returns a new string object since strings are immutable.

Q4. What is the time complexity of inserting at the beginning of a Python list vs a deque?

Ans: Inserting at the beginning of a Python list with list.insert(0, x) is O(n) because all existing elements must be shifted one position to the right to make room. deque.appendleft(x) is O(1) because a deque is implemented as a doubly-linked list, allowing constant-time insertion and deletion at both ends without shifting.

Q5. How does Python's list dynamically resize?

Ans: Python lists use a dynamic array (over-allocation) strategy. When the internal buffer is full, Python allocates a new, larger buffer (roughly 1.125x growth factor) and copies all existing elements into it. This makes individual appends O(n) in the worst case but O(1) amortised over many appends, since resizing happens infrequently. The list object tracks both the logical length (len()) and the allocated capacity separately.